RoadBook design

This document is for people who want to read or change the source. For how to use the program, see the user manual.

RoadBook is a single-process desktop application written in Python with PySide6 (Qt 6). There is no server and no database: a roadbook is a directory of CSV files that is loaded into memory as Python dataclasses, edited through Qt table models, and written back on save.

Contents

1. Running and building
2. Source map
3. Data model
4. Storage
5. The main window
6. Tables: ObjectTable and ObjectTableModel
7. Keeping names coherent
8. Trades, graphs and OCR
9. Calendar
10. Graphics window
11. Filters and statistics
12. Menus, toolbar and settings
13. Conventions
14. Where to make common changes

Running and building

```
pip install -r requirements.txt
python3 rb.py                 # DEBUG=y prints OCR diagnostics
pyinstaller --noconfirm rb.spec
```

`.github/workflows/python-app.yml` builds executables for Linux, Windows and macOS with PyInstaller when run by hand on GitHub.

The program has no test suite in the repository. Qt code can be tested without a display by setting `QT_QPA_PLATFORM=offscreen`, creating a `QApplication`, building the windows and calling their methods or sending events with `QtTest`.

Source map

Files are grouped here by layer. Arrows in the import graph only go downward: the data layer knows nothing about Qt widgets.

Layer	File	What it holds
Entry	`rb.py`	finds roadbooks on disk, creates `DataWindow`, opens the last roadbook used
Main window	`datawin.py`	`DataWindow` (docks, wiring, open and save), `TradeEdit` (trade form), `AccountEdit`, `FileDropLineEdit`
	`menus.py`	`MainMenus`: menu bar, toolbar, recent roadbooks (in `QSettings`)
	`icons.py`	inline SVG icons rendered to `QIcon`
Widgets	`objtbl.py`	`ObjectTable` and `ObjectTableModel`, a generic editable table over dataclasses, with grouping, find and replace, highlighting
	`calwin.py`	`TradeCalendar`, `DayCalendar` and `YearView`
	`imgwin.py`	`ImageViewer`, the graphics window
	`filterwin.py`	`FilterWindow`
	`statswin.py`	`StatsWindow` and `TodayPanel`
	`checklist.py`	`CheckBoxGroup`, a list of check boxes
	`find.py`	`FindReplaceDialog`
	`plot.py`, `bar.py`, `stackbar.py`, `pie.py`	chart widgets built on Qt Charts
Logic	`stats.py`	`Filter`, `StatTotals`, the per-day/week/setup/... aggregations and units
	`graphocr.py`	`TradeOCR`: reads trade data from a chart screenshot (EasyOCR and OpenCV, loaded lazily)
Data	`data.py`	the dataclasses, `RoadBook`, the `TABLES` description, graph file naming
	`csv_mapper.py`	generic dataclass ↔ CSV mapping with atomic writes
	`newdata.py`	the CSV contents of a new roadbook

`cotasfvg.py` is a stand-alone helper and is not used by the application.

Data model

All in `data.py`. Each table row is a `@dataclass`; the field order is the CSV column order.

Class	Key field	Notes
`Account`	(one row)	account size, `neutral` threshold, `fixed`, `copygraphs`, format `version`
`Currency`	`name`	`euros2`: value in euros, used to convert results
`Instrument`	`instrument`	`currency`, `pipmoney` (money per point), `scale` (to normalize points), and other per-instrument numbers
`Setup`	`setup`	name and description
`Feature`	`feature`	`setups`: the setups it applies to; empty means all
`Trade`	`trade` (number)	instrument, setup, date and times, `dir`, `lots`, `tp`, `stop`, `out` (points), `euros`, `graf` (list of graph paths, primary first), `notes`, `has` (set of feature names)
`Trade1`		the version 1 trade format, converted with `Trade.old2new` on load

Trades reach the roadbook through `t.rb` (set after loading), which they use to find their instrument and currency:

- `Trade.points()` is `out`; `Trade.ptseuros()` is `euros` when set, else the points times lots, money per point and currency rate.
- `Trade.result(neutral)` returns `Result.OK`, `KO` or `Neutral` by comparing euros (or points) with the account's `neutral` value. The same rule colours rows, groups and calendar days.
- `Trade.checkOut()` validates a trade and returns an error text or `None`.

`RoadBook` holds the lists (`trades`, `setups`, `features`, `instruments`, `currencies`), the `account`, `dir`, `dirty`, `filteredtrades` (the result of the current filter, or `None`) and the next trade number. It also has the lookups (`findInstrument`, `setupNames`, `featureNames(setup)`...), the "is this name used" checks for deletes, and the rename and defaults helpers described below.

Storage

A roadbook is a directory:

```
account.csv  trades.csv  setups.csv  features.csv  activos.csv  divisas.csv
diarygraphs/trade12dax.png  diarygraphs/trade12dax.1.png  ...
```

`TABLES` in `data.py` describes each list-like CSV once: the `RoadBook` attribute, the file name, the dataclass, the sort key applied when loading, and column renames (trades store `datein` as `date` and `has` as `with`). `RoadBook.load()` and `save()` loop over it, so adding a table means adding one `Table(...)` line plus its dataclass.

`csv_mapper.py` maps dataclasses to CSV by their type hints:

- `;` is the column separator, and `set[str]` and `list[str]` cells are `;`-joined too. Sets are sorted when written; lists keep their order.
- `date`, `time`, `Enum`, `bool`, numbers and `Optional[...]` are supported. An empty cell for a field with a default takes the default.
- `load_objects_from_csv` returns `(objects, errors)` and does not stop at a bad row.
- `write_objects_to_csv` writes `file.tmp`, flushes and `fsync`s it, then `os.replace`s it over the target, so a crash never leaves half a file. Before that, `RoadBook.savetable` copies the old file to `file~`.

Format migration happens in `RoadBook.loadtable`: when `account.csv` has a `version` older than `VERSION`, the trades are read as `Trade1` and converted with `Trade.old2new`; saving then writes the new version. `canonlist` repairs graph lists that older versions saved sorted by name.

`RoadBook.mknew(dir)` writes the template files from `newdata.py` (`newdata.py` has CRLF line endings; keep them).

The main window

`DataWindow` (`datawin.py`) is a `QMainWindow` with no central widget; all content is in dock widgets:

- Left, tabbed: Trades, Calendar, Setups, Features, Instruments, Currencies, Account (`self.leftdocks`).
- Right, tabbed: Trade Features, Feature Setups, Today (`self.rightdocks`). `shrinkrightdock()` gives them their minimum width at start.

`changedata(rb)` installs a roadbook: it hands each list to its table, refreshes the panels and calendar, opens the latest trade groups (`initialgroups`) and records the roadbook as recent.

Components talk through **Qt signals**, never callbacks passed in constructors. The main ones:

Signal	From	Handled by
`selected(obj)`	each `ObjectTable`	`selectedtrade` / `selectedfeature`: right panels, calendar, graphics
`dirtied()`	tables, check box groups	`DataWindow.dirtied`: marks the roadbook unsaved
`edited()`, `removed(obj)`	tables (rows added or deleted)	`dirtiedTrades`, `refreshfilter`
`renamed(old, new)`	name tables	`renamedSetup`, `renamedFeature`...
`fieldChanged(obj, name)`	tables	`tradeFieldChanged`, `nameFieldChanged`
`graphicsRequested(obj)`	trades table	`tradegraphics`
`filterChanged(flt)`	`FilterWindow`	`setfilter`
`moved(delta)`, `jumped(last)`	`ImageViewer`	trades table `next` / `prev` / `edge`
`tradeSelected`, `tradeActivated`, `monthChanged`	`TradeCalendar`	`calendartrade`, `calendaropen`, `calendarmonth`
`changed()`	`TradeEdit`	`dirtiedTrades`

`dirtiedTrades()` is the "trades changed" hub: it marks the roadbook dirty and refreshes the Today panel, the totals label, the calendar and the stats window.

Tables: ObjectTable and ObjectTableModel

`objtbl.py` is the most reused piece. `ObjectTableModel` is a `QAbstractTableModel` over a list of dataclass objects; the columns are the dataclass fields (in an optional `order`), shown and edited as text through `csv_mapper.format_value` and `convert_value`. `ObjectTable` is the widget: the view, the buttons (New, Del, Edit, Find, plus optional Graphics, Filter, Stats), the info label and the find dialog.

Options passed by `DataWindow`:

- `rdonly`: columns that can't be edited.
- `uniquekey`: column 0 is a unique, non-empty name; changing it emits `renamed(old, new)` *before* the value changes, so the roadbook can still find things by the old name.
- `editor(obj, filepath)`: a dialog used to create and edit rows (the trade form) instead of in-place creation.

- `canremove(obj)`: vetoes deletes.
- `groupby`, `sortkey`, `groupinfo`: turn on grouping (used for trades).

Grouping

With `groupby` set, the model keeps two extra lists:

- `order`: all objects sorted by `sortkey`.
- `rows`: what the view shows, a mix of `Group` header rows and objects. Objects inside a collapsed group are left out.

`groupby(obj)` returns the object's groups from the outside in, as `(key, label)` pairs with keys unique across levels; for trades that is `(year,)`, `(year, month)`, `(year, month, isoweek)`. `collapsed` is a set of group keys, and `build()` recomputes `rows` from `objects` and `collapsed`.

Rules to keep in mind when touching the model:

- **Never index `objects` by view row.** Use `objat(row)` (an object or `None` for a header), `entry(row)`, `rowof(obj)`, `isgroup(row)`.
- Structural changes (`insertRows`, `removeRows`, `toggle`, `collapseexcept`, `changedata`) rebuild through `regroup()`, which is a model reset. `ObjectTable` saves the current object and scroll position in `modelAboutToBeReset` and restores them, plus the header spans and the corner triangle, in `modelReset`.
- `reveal(obj)` expands the groups holding `obj` and returns its row; use it before selecting an object that may be hidden (`ObjectTable.selectobj`).
- Navigation (`next`, `prev`, `edge`) and find walk `ordered()`, so they reach trades inside collapsed groups.
- Editing a value that changes an object's place (date or sort key) defers a `regroup()` with `QTimer.singleShot(0, ...)`, so the reset does not happen under the editor that is committing the value.

Without `groupby`, `rows` and `order` are `None` and the model is a plain list, which is what the setups, features, instruments and currencies tables use.

Find, replace and highlighting

`ObjectTableModel.find(index, pattern, backward)` searches cell text object by object, wrapping around. `replace` goes through `setData`, so replacing runs the same validation and signals as typing. `replaceall` uses `setvalue` directly and reports how many cells changed and how many were refused. `HighlightDelegate` paints the matches of the current pattern in yellow and the current cell in orange.

Each table's Ctrl+F shortcut only works while that table has the focus; `MainMenus` turns them off and gives Ctrl+F to **Edit ▸ Find**, which picks the table that is shown (`DataWindow.findintable`). Several window-wide shortcuts for the same key would cancel each other in Qt.

Keeping names coherent

Trades refer to setups, features and instruments by name, features refer to setups, and instruments refer to currencies. Two mechanisms keep those references valid:

- **Renames.** A name table emits `renamed(old, new)`, and `DataWindow` calls `RoadBook.rensetup`, `renfeature`, `reninstrument` or `rencurrency`, which rewrite every reference.
- **New names.** When a trade field (`instrument`, `setup`, `has`) or a feature's `setups` or an instrument's `currency` is set to a name the roadbook doesn't have, `tradeFieldChanged` / `nameFieldChanged` call the `default*` helpers (`defaultinstr`, `defaultsetup`, `defaultfeatures`...), which add a default entry, and `refreshnames()` shows it.

Deletes of names in use are vetoed by `canremove` with the `*Used` checks.

Trades, graphs and OCR

`TradeEdit` is the trade form. It turns its fields into a `Trade` in `edited()`, validates it with `checkOut()`, copies it into the existing trade (`copy_from`) when editing, stores new graphs and fills in missing names with `defaultsfortrade`.

Graphs:

- `RoadBook.mkgraphpath(t, k)` names them `diarygraphs/trade<N><instr>.png` (k = 0, the primary graph) and `trade<N><instr>.<k>.png`; `graphindex` reads k back.
- `RoadBook.storegraphs(t)` copies the graphs of a trade that are not yet in `diarygraphs/` to the first free name, keeping their order and never overwriting a file. Graphs already there are left alone.
- `graphpaths(t)` returns the paths to show, and `findgraphs` / `savegraphs` handle Save As.

OCR (`graphocr.py`): `TradeOCR.OCR(path, setups)` runs EasyOCR on a screenshot and extracts the instrument, setup, direction, TP and stop points, success, size, date and times. EasyOCR and OpenCV are imported on first use, because importing them takes seconds. When several files are dropped on a trade with no graphs, `TradeEdit.ocrbest` scores each one (`ocrscore`: a known instrument counts most) and makes the best the primary graph.

Calendar

`calwin.py`:

- `DayCalendar` is a `QCalendarWidget` that paints its own cells (`paintCell`) from a `date → colour` map, so day colours are not hidden by the selection. The selected day gets a frame instead. With `onemonth=True` it shows a single month and no navigation (used by the year view).
- `YearView` holds twelve `DayCalendar`s in a scroll area and reflows them to the available width.
- `TradeCalendar` combines a month and a year view. `settrades(trades, neutral)` groups the trades per day and colours each day by its total. It emits `tradeSelected` and `tradeActivated` with the first trade of a clicked day, and `monthChanged` when *the user* moves to another month. `showdate()` moves it when a trade is selected elsewhere, with `quiet` set so that does not emit `monthChanged`.

`tradeorder` (date, time, number) is the trade sort key shared by the calendar and the trades table.

Graphics window

`ImageViewer` (`imgwin.py`) shows one graph of a list at a time. `mode` is `"fit"`, `"real"` or `"zoom"` and is kept across graphs and trades; the fit scale uses the viewport size without scroll bars. Double clicks cycle through `dstep` (zoom ×2, ×2 again, fit), and `zoomat` keeps the clicked point under the mouse. The arrow, Home and End keys are `QShortcut`s so the scroll area does not take them. It never selects trades itself: it emits `moved(delta)` and `jumped(last)`, and the main window selects the trade, which in turn calls `setimage()` back.

Filters and statistics

`stats.Filter` is a dataclass of criteria (feature sets that must and must not be present, instruments, setups, directions, results, hours, weekdays, dates); `apply(trades)` returns the matching ones. `FilterWindow` builds one from its check boxes and emits it. `DataWindow.setfilter` stores the result in `rb.filteredtrades`, shows it in the trades table and refreshes the calendar and statistics.

`stats.py` computes values per trade in a `StatUnit` (`tradevalue`), totals (`StatTotals`) and aggregations per day, week, month, setup, instrument, weekday and hour. `StatsWindow` lays out the chart widgets for the selected plots, unit and kinds (total, average, count), and `TodayPanel` shows today's pies.

Menus, toolbar and settings

`MainMenus` (`menus.py`) builds the menu bar and toolbar from `QAction`s created by `mkaction`, with platform-standard shortcuts (`QKeySequence.StandardKey`) and menu roles so Quit and About land in the macOS application menu.

Recent roadbooks are stored with `QSettings("pylg", "RoadBook")` under `recent` (newest first, at most 10). `addrecent` is called from `DataWindow.changedata`; `updaterecent` rebuilds the toolbar's roadbook buttons, and the recent menu is filled each time it opens. `rb.py` opens the first recent roadbook that still exists.

Icons are SVG strings in `icons.py` rendered by `QSvgRenderer`, so they look the same on every platform.

Conventions

- Tabs for indentation, short lowercase method names (`mkfilter`, `refreshnames`), docstrings for anything not obvious.
- Talk between components with signals declared at class level (`selected = Signal(object)`); widgets that wrap a model re-emit the model's signals.
- Data classes stay free of Qt. Anything shown in a table is a dataclass.
- Explicit imports only (no `import *`).
- Write files atomically through `csv_mapper`; never write a CSV in place.

Where to make common changes

To...	Change
add a field to trades	the `Trade` dataclass (and `copy_from`, `old2new` if relevant), `TRADEVIEWORDER`, the `TradeEdit` form and `newdata.py`
add a new kind of named table	a dataclass, a `Table` line in `TABLES`, a template in `newdata.py`, an `mknamestbl` call and dock in `DataWindow`
add a statistic	an aggregation in `stats.py`, a `StatPlot` value and its layout in `StatsWindow.mkstats`
add a filter criterion	a field in `stats.Filter` and its check in `apply`, and a control in `FilterWindow.getFilter`
add a menu entry or shortcut	`MainMenus.mkmenubar`
change what a trade's colour means	`Trade.result`, which the table, groups and calendar all use